Session: Recurrent Neural Networks

Javier Serrano

Applied Machine Learning
Master in Data Science and Analytics

Strathmore University

@iLabAfrica Centre

Processing Sequences using RNN

RNN = Recurrent Neural Networks

- Predicting the future with a class of networks call RNN
- RNN can work on sequences of arbitrary lengths (a difference from CNN).
- Useful with time series, sentences, documents, audio, video
- Apart from RNN, for small sequences, a regular dense network and for very long sequences, such as audio samples or text, convolutional neural networks can work quite well.

Sumary

Recurrent Neurons and Layers

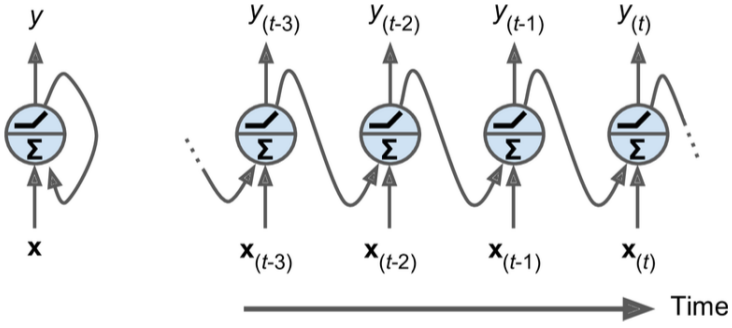
Training RNNs

Forecasting Time Series

Handling Long Sequences

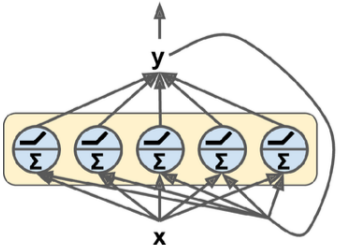
Recurrent Neurons

1. So far, we studied feedforward neural networks
2. In a recurrent neural network it has connections pointing forward and backward

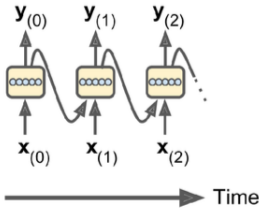


A recurrent neuron unrolling the network through time

A Layer of Recurrent Neurons



A layer of recurrent neuron



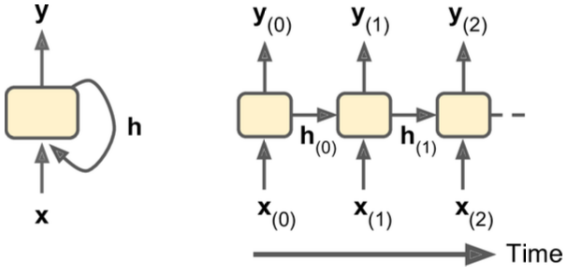
unrolled through time

$$\mathbf{Y}_{(t)} = \phi\left(\mathbf{X}_{(t)}\mathbf{W}_x + \mathbf{Y}_{(t-1)}\mathbf{W}_y + \mathbf{b}\right)$$
$$= \phi\left(\left[\mathbf{X}_{(t)} \quad \mathbf{Y}_{(t-1)}\right]\mathbf{W} + \mathbf{b}\right) \text{ with } \mathbf{W} = \begin{bmatrix}\mathbf{W}_x \\ \mathbf{W}_y\end{bmatrix}$$

Output of a layer of recurrent neurons for a minibatch

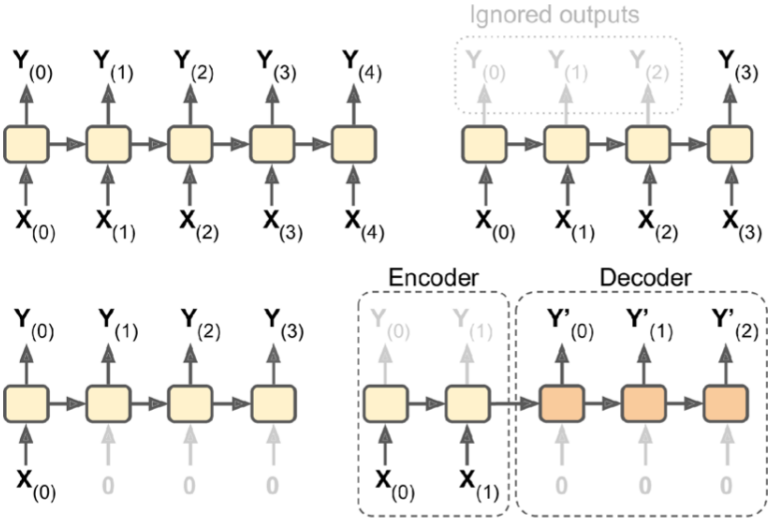
Memory Cells

- The output of a recurrent neuron at time step t is a function of all the inputs from previous time steps, **MEMORY**
- In the case of the basic cell, the output is simply equal to the state.
- But the standard structure is...



Input and Output sequences

Sequence to Sequence, Sequence to Vector, Vector to Sequence, Encoder-Decoder



Sumary

Recurrent Neurons and Layers

Training RNNs

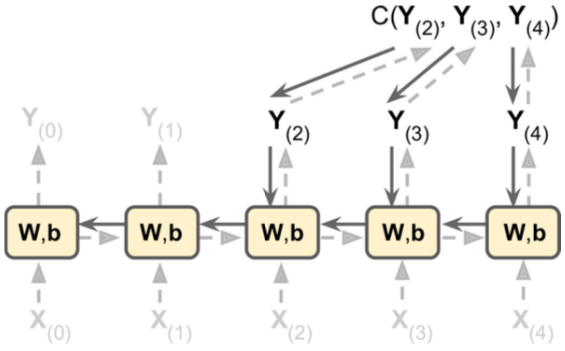
Forecasting Time Series

Handling Long Sequences

Training RNNs

Backpropagation through time

- The trick is to unroll it through time and then simply use regular backpropagation
- tf.keras takes care of all of this complexity



Sumary

Recurrent Neurons and Layers

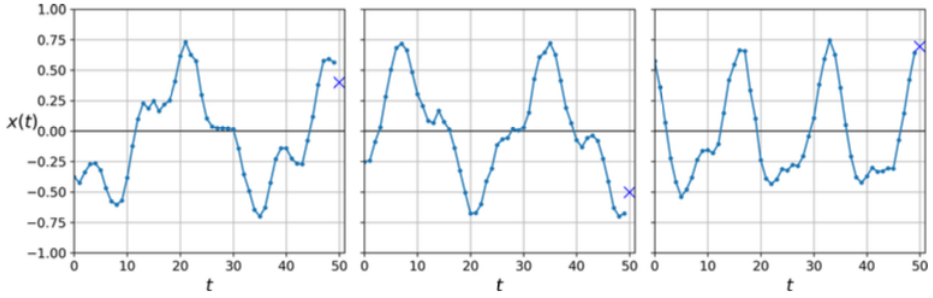
Training RNNs

Forecasting Time Series

Handling Long Sequences

What Time Series are?

- A sequence of one or more values per time step.
- Univariate vs Multivariate time series
- Two possible tasks: Forecasting vs imputation of missing values



One solution: A fully connected network

- Let's use a simple Linear Regression model so that each prediction will be a linear combination of the values in the time series

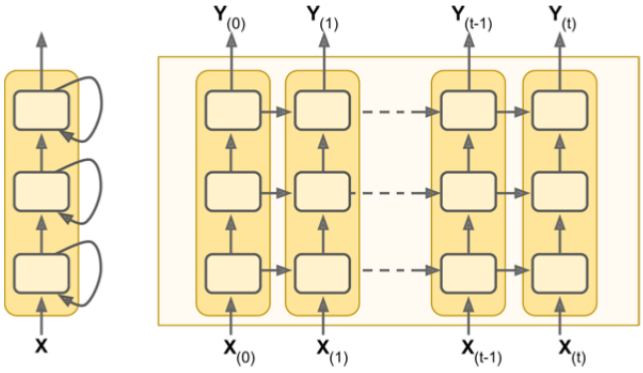
```
model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[50, 1]),
    keras.layers.Dense(1)
])
```

- Xe compile this model using the MSE loss and the default Adam optimizer, then fit it on the training set for 20 epochs and evaluate it on the validation set, we get an MSE of about 0.004. **Not bad**.

A Better solution: A Deep RNN

Architecture

- We do not need to specify the length of the input sequences
- Uses the hyperbolic tangent activation funtion
- The initial state $h_{(init)}$ is set to 0



A Better solution: A Deep RNN

Keras Implementation

- A part from SimpleRNN we can use other cells as LSTM or GRU
- We can substitute the last layer for an Dense one, so you can use any activation fuction.

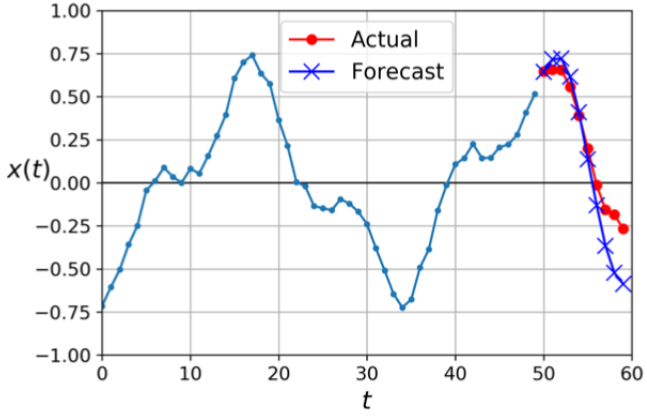
```
model = keras.models.Sequential([
    keras.layers.SimpleRNN(20, return_sequences=True, input_shape=[None, 1]),
    keras.layers.SimpleRNN(20, return_sequences=True),
    keras.layers.SimpleRNN(1)
])
```

-

Forecasting Several Time Steps Ahead

Option 1

1. Use the model we already trained. Prediction are worse for later time steps



Forecasting Several Time Steps Ahead

Option 2

2 The second option is to train an RNN to predict all 10 next values at once. Work nicely

```
model = keras.models.Sequential([
    keras.layers.SimpleRNN(20, return_sequences=True, input_shape=[None, 1]),
    keras.layers.SimpleRNN(20),
    keras.layers.Dense(10)
])
```

Forecasting Several Time Steps Ahead

Option 3

3 Turn the model into a sequence-to-sequence model. *TimeDistributed* layer for this very purpose: it wraps any layer (e.g., a Dense layer) and applies it at every time step of its input sequence. Best forecasting

```
model = keras.models.Sequential([
    keras.layers.SimpleRNN(20, return_sequences=True, input_shape=[None, 1]),
    keras.layers.SimpleRNN(20, return_sequences=True),
    keras.layers.TimeDistributed(keras.layers.Dense(10))
])
```

Sumary

Recurrent Neurons and Layers

Training RNNs

Forecasting Time Series

Handling Long Sequences

Handling Long Sequences

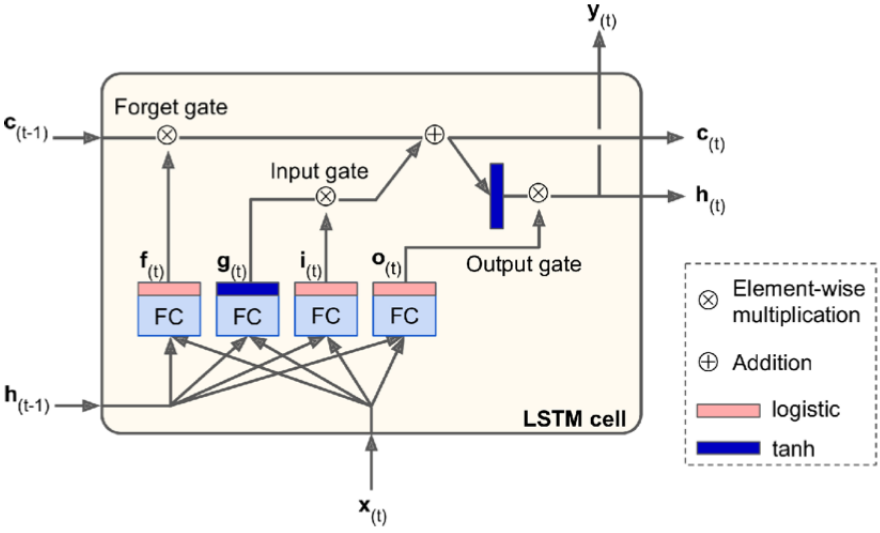
Two problems

1. **Unstable Gradients.** To train an RNN on long sequences, we must run it over many time steps, making the unrolled RNN a very deep network.
   - As usual: good parameter initialization, learning rates, faster optimizers, dropout.
   - Non saturating activation function does not works
   - Neither Batch Normalization, cannot be done between time steps.
   - Sometimes **Layer Normalization** works
2. **Short Term Memory Problem.** When an RNN processes a long sequence, it will gradually forget the first inputs in the sequence.

Short-Term Memory Problem

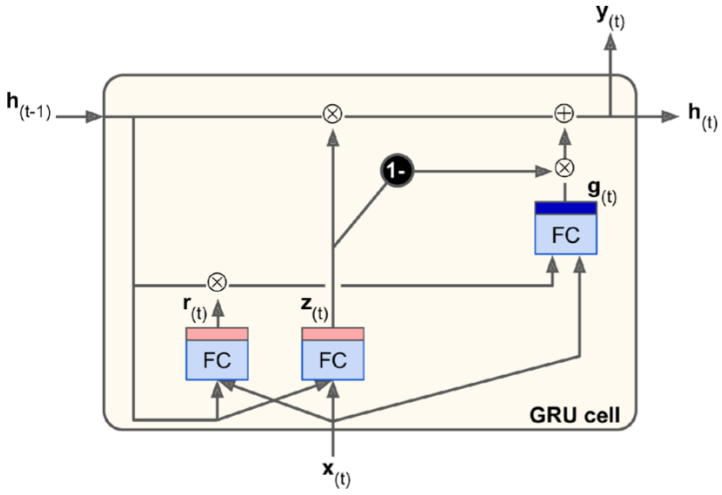
- When traversing an RNN, some information is lost at each time step.
- After a while, the RNN's state contains no trace of the first inputs.
- Various types of cells with long-term memory have been introduced:
  - The *Long Short-Term Memory* (LSTM) cell
  - The Gated Recurrent Unit (GRU) cell
- This two work well with not very long sequences (up to 100 time steps)

The LSTM cell



The GRU cell

A simplified version of the LSTM cell



1D convolutional layers to process long sequences

Example: audio samples, long time series, or long sentences.

- 1D convolutional layer slides several kernels across a sequence as with 2D convolutional layers.
- It produce a 1D feature map per kernel.
- Each kernel will learn to detect a single very short sequential pattern (no longer than the kernel size)
- We can build a neural network composed of a mix of recurrent layers and 1D convolutional layers
- The 1D convolutional layer compress the sequence and the GRU can detect longer patterns

Exercise

Which neural network architecture could you use to classify videos?

Exercise

Which neural network architecture could you use to classify videos?

- Take one frame per second (or whatever)
- Run every frame through the same convolutional neural network (e.g., a pretrained Xception model)
- Feed the sequence of outputs from the CNN to a sequence-to-vector RNN
- Finally run its output through a softmax layer, giving you all the class probabilities.
- For training we could use cross entropy as the cost function.
- If you wanted to use the audio for classification as well, you could use a stack of strided 1D convolutional layers to reduce the temporal resolution from thousands of audio frames per second to just one per second (to match the number of images per second), and concatenate the output sequence to the inputs of the sequence-to-vector RNN (along the last dimension).

Sumary

Recurrent Neurons and Layers

Training RNNs

Forecasting Time Series

Handling Long Sequences

References

- A. Geron, *Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow, O'Reilly*, 2019.
- Andrew Ng course on Deep Learning
  `https://www.youtube.com/watch?v=E13qqHb3J7U`
- I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016.
  `https://github.com/janishar/mit-deep-learning-book-pdf`
- Simon J.D. Prince *Understanding Deep Learning* 2024
  `https://udlbook.github.io/udlbook/`